
PILLAR 7: System Design & Architectural Basics (SDE View)

7.1 Horizontal vs. Vertical Scaling

Clear Definition & Internal Mechanics

Vertical Scaling (Scale Up): Increasing the resources of a single machine — more CPU cores, more RAM, faster NVMe SSDs, faster network interfaces. The application runs on one (more powerful) machine.

Horizontal Scaling (Scale Out): Adding more machines to the pool. The application runs on *multiple* commodity machines. Requests are distributed across them.

Vertical Scaling — Details:

- **Ceiling:** Limited by the largest available machine (e.g., AWS `u-24tb1.metal` with 448 vCPUs and 24 TB RAM — extremely expensive).
- **Zero code changes:** Existing single-threaded or stateful applications can often scale vertically without modification.
- **Single point of failure:** One machine goes down → entire service down. No redundancy.
- **Cost curve:** Non-linear. Doubling CPU often costs 4-8x more due to enterprise hardware pricing.
- **Downtime for upgrade:** Adding RAM/CPU to a bare-metal server typically requires maintenance windows.

Horizontal Scaling — Details:

- **Near-infinite ceiling:** Add more commodity machines as needed. AWS Auto Scaling can provision hundreds of instances within minutes.
- **Requires statelessness:** If application state (sessions, local cache) is stored on one machine, routing requests to different machines produces inconsistent behavior. State must be externalized (Redis, DB, distributed cache).
- **Load Balancer required:** A traffic distributor must route requests across instances.
- **Operational complexity:** Service discovery, distributed tracing, inter-service communication, distributed configuration — all become concerns.
- **Cost:** Efficient at scale; commodity instances cost less per unit of compute than enterprise hardware.

- **High availability:** Instance failure is masked by the other instances. Combined with health checks and auto-replacement.

The Statelessness Requirement:

For horizontal scaling to work correctly, each application instance must treat every request independently. This means:

- **Sessions:** Use distributed session storage (Redis via Spring Session) or JWT tokens (self-contained, no server-side state).
- **Caches:** Use a distributed cache (Redis, Memcached) instead of in-JVM `HashMap`.
- **File uploads:** Use object storage (S3, GCS) instead of local filesystem.
- **Scheduled jobs:** Use a distributed scheduler (Quartz with JDBC store, Spring Batch) to prevent multiple instances running the same job simultaneously.

Dimension	Vertical	Horizontal
Ceiling	Hardware max	Near-unlimited
Redundancy	✗ Single point of failure	✓ N-1 failure tolerance
Complexity	Low (same app)	High (distributed system)
Cost at scale	Expensive (exponential)	Cheaper (linear)
Stateful apps	✓ Works	✗ Requires refactoring
Typical use	DBs, legacy apps	Microservices, APIs

Real-Life Analogy

Vertical scaling is like **upgrading a single worker** — giving your one cashier a faster scanner, two cash drawers, and a headset. There's a hard limit to how fast one person can go. **Horizontal scaling** is like **hiring more cashiers** — each one handles their own line. Now you can handle 10x the customers by opening 10 checkout lanes. But you need a manager (load balancer) to direct customers to the shortest lane.

Interview Context & "Why it Matters"

Almost every system design interview involves a scaling question. Candidates are expected to recommend horizontal scaling for stateless services and understand *why* statelessness is a prerequisite. They should also know that databases typically start with vertical scaling + read replicas before resorting to sharding (horizontal DB scaling).

java

```

import org.springframework.session.data.redis.config.annotation.web.http.EnableRedisHttpSession;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.connection.lettuce.LettuceConnectionFactory;
import org.springframework.session.web.http.CookieSerializer;
import org.springframework.session.web.http.DefaultCookieSerializer;

/**
 * Enables distributed HTTP sessions via Redis.
 * Critical for horizontal scaling: any instance can serve any session.
 * Replaces in-memory HttpSession with Redis-backed SessionRepository.
 *
 * Without this, a user logged into instance A gets a 401 if routed to instance B.
 * With this, all instances share the same session store.
 *
 * Dependencies:
 *   spring-session-data-redis
 *   spring-boot-starter-data-redis
 */
@Configuration
@EnableRedisHttpSession(maxInactiveIntervalInSeconds = 1800) // 30 min TTL
public class DistributedSessionConfig {

    @Bean
    public RedisConnectionFactory redisConnectionFactory() {
        // In production: LettuceConnectionFactory with RedisClusterConfiguration
        // or RedisSentinelConfiguration for high availability
        return new LettuceConnectionFactory("redis-host", 6379);
    }

    @Bean
    public CookieSerializer cookieSerializer() {
        DefaultCookieSerializer serializer = new DefaultCookieSerializer();
        serializer.setCookieName("SESSION");
        serializer.setUseSecureCookie(true); // HTTPS only
        serializer.setUseHttpOnlyCookie(true); // Prevent XSS cookie theft
        serializer.setSameSite("Strict"); // CSRF protection
        return serializer;
    }
}

// — Stateless JWT alternative (no server-side session state) —

```

```

/**
 * JWT-based authentication – fully stateless.
 * Each token is self-contained: user ID + roles + expiry.
 * Any instance can validate the token without consulting a shared store.
 * Tradeoff: Cannot invalidate individual tokens before expiry (need blacklist or sh
 */
class JwtTokenValidator {

    private final String secret;
    private static final long EXPIRY_MS = 15 * 60 * 1000; // 15 minutes

    JwtTokenValidator(String secret) {
        if (secret == null || secret.length() < 32) {
            throw new IllegalArgumentException("JWT secret must be at least 32 chara
        }
        this.secret = secret;
    }

    // In a real implementation, use io.jsonwebtoken:jjwt-api library:
    // Claims claims = Jwts.parserBuilder().setSigningKey(key).build().parseClaimsJw
    public boolean validate(String token) {
        if (token == null || token.isBlank()) return false;
        // Parse and validate signature, expiry, issuer
        // Return true if valid; false if expired, tampered, or malformed
        return true; // simplified for illustration
    }
}

```

Common Interview Question & Expert Answer

Q: "Your API server currently runs on a single large EC2 instance. How would you transition it to a horizontally scaled architecture? What are the key dependencies you must address?"

A: I'd address dependencies in this order:

- 1. Session Externalization:** Replace in-JVM `HttpSession` with Spring Session + Redis. All instances now share session state. Deploy: spin up a Redis cluster with replica nodes for HA.
- 2. Local Cache Externalization:** Any `@Cacheable` backed by in-memory Caffeine needs to be replaced with Redis (`spring-boot-starter-data-redis` + `@EnableCaching`). Otherwise, cache invalidation on instance A doesn't propagate to instance B.
- 3. Scheduled Job Coordination:** Any `@Scheduled` jobs need ShedLock or Quartz with a JDBC/Redis lock store. Without this, all N instances run the job simultaneously — causing

duplicates, race conditions, or duplicate emails.

4. File Storage Migration: Any file uploads saved to the local filesystem (`/tmp`) must move to S3 or GCS. Otherwise, a file uploaded to instance A is inaccessible from instance B.

5. Load Balancer + Health Checks: Put an ALB/NLB in front. Configure `/actuator/health` as the health check endpoint. Spring Boot Actuator provides this automatically. The LB removes unhealthy instances from rotation and adds new ones after passing health checks.

6. Configuration Externalization: Any hardcoded environment-specific config (DB URLs, API keys) must be in environment variables, AWS Parameter Store, or Spring Cloud Config Server — not baked into the JAR.

7. Distributed Tracing: With multiple instances, request traces span multiple logs. Add Spring Cloud Sleuth + Zipkin/Jaeger to correlate traces across instances via a shared `X-B3-TraceId` header.

Once all dependencies are addressed, deploy behind an Auto Scaling Group with a target tracking policy (e.g., scale out when CPU > 70% for 3 minutes).

7.2 Load Balancers (Nginx, AWS ALB)

Clear Definition & Internal Mechanics

A **Load Balancer (LB)** is a network component that distributes incoming client requests across multiple backend server instances. It acts as the single entry point for clients while hiding the fleet of backend instances.

Load Balancing Algorithms:

Algorithm	Mechanism	Best For
Round Robin	Requests distributed sequentially: S1, S2, S3, S1, S2...	Homogeneous servers, uniform request cost
Least Connections	Route to server with fewest active connections	Varying request durations
IP Hash	Hash client IP → always routes to same server	Session stickiness without Redis
Weighted Round Robin	Servers assigned weights; higher weight = more requests	Mixed instance types
Random	Random server selection	Simple, low-overhead

Layer 4 vs. Layer 7:

- **Layer 4 (Transport):** Routes based on TCP/UDP IP:port. Fast, no HTTP awareness. AWS NLB. Cannot route based on URL path or headers.
- **Layer 7 (Application):** Routes based on HTTP headers, URL path, cookies, request body. AWS ALB, Nginx. Enables path-based routing (`/api/*` → API cluster, `/static/*` → CDN), header-based routing, SSL termination, WebSocket support.

Health Checks: LBs periodically check each backend instance (HTTP GET `/health` or TCP connect). Instances that fail health checks are removed from rotation. When they recover, they're added back. This enables zero-downtime deployments (rolling updates).

AWS ALB Specifics:

- **Target Groups:** Groups of EC2 instances, ECS tasks, or Lambda functions. ALB routes to target groups based on rules.
- **Listener Rules:** Path-based (`/api/*`), host-based (`api.example.com`), header-based routing.
- **Sticky Sessions (Session Affinity):** ALB can route a client to the same target using a cookie (`AWSALB`). Avoid this when possible — use distributed sessions instead.
- **SSL Termination:** ALB decrypts HTTPS, forwards HTTP to backends (reduces TLS overhead on app servers, centralizes certificate management).

Nginx as Reverse Proxy:

nginx

```

upstream api_cluster {
    least_conn; # Least connections algorithm
    server app1:8080 weight=3; # Weighted: gets 3x traffic of app2
    server app2:8080 weight=1;
    server app3:8080 backup; # Only used when app1 and app2 are down

    keepalive 32; # Maintain 32 persistent connections to each upstream
}

server {
    listen 443 ssl;
    ssl_certificate /etc/ssl/certs/api.crt;
    ssl_certificate_key /etc/ssl/private/api.key;
    ssl_protocols TLSv1.2 TLSv1.3;

    location /api/ {
        proxy_pass http://api_cluster;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header Host $host;
        proxy_connect_timeout 5s;
        proxy_read_timeout 30s;
    }

    location /health {
        access_log off; # Don't log health check noise
        return 200 "OK";
    }
}

```

Real-Life Analogy

A load balancer is a **traffic cop at a busy intersection** managing cars entering a parking garage with multiple entry lanes. The cop (LB) stands at the single entrance, checks which lanes are less busy (least-connections), and waves each car to an available lane. If a lane is under construction (health check failure), the cop stops routing to it. When it reopens, they route normally again.

Interview Context & "Why it Matters"

Load balancers are the foundation of high availability. Interviewers expect candidates to know: the difference between L4 and L7, when to use each, health check configuration, and the implications of session stickiness.

Production-Ready Code Snippet

java

```

import org.springframework.boot.actuate.health.Health;
import org.springframework.boot.actuate.health.HealthIndicator;
import org.springframework.stereotype.Component;
import org.springframework.boot.actuate.endpoint.web.annotation.RestControllerEndpoint;
import org.springframework.web.bind.annotation.GetMapping;

/**
 * Production-ready health check endpoint for load balancer health probes.
 * Spring Boot Actuator exposes /actuator/health automatically.
 * AWS ALB, Nginx, Kubernetes readiness probes all use this endpoint.
 */
@Component
public class ApplicationHealthIndicator implements HealthIndicator {

    private final javax.sql.DataSource dataSource;

    public ApplicationHealthIndicator(javax.sql.DataSource dataSource) {
        this.dataSource = dataSource;
    }

    @Override
    public Health health() {
        // Check DB connectivity – if DB is unreachable, mark unhealthy so LB stops
        try (java.sql.Connection conn = dataSource.getConnection()) {
            boolean valid = conn.isValid(2); // 2-second timeout
            if (valid) {
                return Health.up()
                    .withDetail("database", "reachable")
                    .withDetail("instance", getInstanceId())
                    .build();
            } else {
                return Health.down()
                    .withDetail("database", "connection invalid")
                    .build();
            }
        } catch (java.sql.SQLException e) {
            return Health.down(e)
                .withDetail("database", "unreachable")
                .withDetail("error", e.getMessage())
                .build();
        }
    }
}

```

```

private String getInstanceId() {
    // In AWS: fetch from EC2 instance metadata
    // return System.getenv("EC2_INSTANCE_ID");
    return java.net.InetAddress.getLoopbackAddress().getHostName();
}
}

// — Graceful Shutdown (Critical for Zero-Downtime Deploys) —————

/**
 * Spring Boot 2.3+ has built-in graceful shutdown:
 * server.shutdown=graceful
 * spring.lifecycle.timeout-per-shutdown-phase=30s
 *
 * Flow on SIGTERM (sent by orchestrator on redeploy):
 * 1. LB stops routing new requests to this instance (via health check failure or de
 * 2. JVM receives SIGTERM
 * 3. Graceful shutdown waits for in-flight requests (up to timeout)
 * 4. Spring context closes (@PreDestroy hooks run)
 * 5. JVM exits cleanly
 *
 * Configure in application.yaml:
 */
class GracefulShutdownConfig {
    // Equivalent YAML configuration:
    // server:
    //   shutdown: graceful
    // spring:
    //   lifecycle:
    //     timeout-per-shutdown-phase: 30s
    //
    // In AWS ECS/EKS, set terminationGracePeriodSeconds=60 to match.
}

```

Common Interview Question & Expert Answer

Q: "What is the difference between a Layer 4 and Layer 7 load balancer? When would you use each?"

A:

Layer 4 (Transport Layer — TCP/UDP):

- Routing decisions based on IP address and port only — no HTTP awareness.

- Does NOT inspect the request body or headers.
- Extremely fast; minimal processing overhead.
- Supports non-HTTP protocols (TCP, UDP, SMTP, custom binary protocols).
- Example: AWS NLB, HAProxy in TCP mode.
- Use when: ultra-low latency is critical, you need raw TCP passthrough (WebSocket with minimal overhead, gaming servers, financial trading), or you use non-HTTP protocols.

Layer 7 (Application Layer — HTTP/HTTPS):

- Routing decisions based on HTTP method, URL path, headers, cookies, request body.
- Can: terminate SSL, decompress requests, add security headers, route `/api/*` to one cluster and `/admin/*` to another.
- Slightly higher latency (must parse HTTP to make routing decisions).
- Example: AWS ALB, Nginx, Envoy proxy.
- Use when: you need path-based routing, host-based routing, SSL termination, WebSocket upgrade handling, or advanced traffic management (A/B testing via header-based routing, canary deployments).

My default recommendation: Use ALB (L7) for HTTP microservices — path/host-based routing, SSL termination, and native health checks justify the marginal latency increase. Reserve NLB (L4) for non-HTTP services or scenarios requiring static IPs and sub-millisecond routing.

7.3 Database Scaling — Read Replicas & Sharding

Clear Definition & Internal Mechanics

Read Replicas

A **read replica** is a database instance that continuously **replicates** from the primary (master) database. It receives all writes asynchronously (or synchronously for synchronous replication) from the primary and maintains an up-to-date copy.

Replication Mechanism (PostgreSQL example):

- Primary writes to **WAL (Write-Ahead Log)**.
- WAL Sender process streams WAL records to the replica.
- Replica's WAL Receiver applies the records, updating its own data files.
- **Replication lag:** The delay between a write on primary and its visibility on the replica. Typically milliseconds; can grow under high write load.

Traffic Pattern:

- **All writes** → Primary
- **Read queries** (reports, search, user profile reads) → Replica(s)

Benefits:

- Offloads 70–90% of typical read traffic from the primary.
- Replicas can be optimized for reads (different index configurations, read-optimized query settings).
- Replicas can serve as **hot standby** — promoted to primary in seconds on primary failure (AWS RDS Multi-AZ does this automatically).

Limitations:

- **Replication lag**: A replica may serve stale data. For session-sensitive reads (e.g., "show me the order I just placed"), route to primary.
- Does NOT help with write throughput. All writes still go to one primary.

Database Sharding

Sharding horizontally partitions data across multiple databases (shards). Each shard is an independent database containing a subset of the data.

Shard Key Selection (Critical): The shard key determines which shard stores each record. A poor shard key causes **hot spots** (one shard overloaded; others idle).

Shard Key Strategy	Mechanism	Pros	Cons
Range-based	User IDs 1–1M → Shard 1, 1M–2M → Shard 2	Simple, range queries efficient	Hot spots if new users flood one shard
Hash-based	$\text{shard} = \text{hash}(\text{userId}) \% \text{numShards}$	Even distribution	Range queries require all-shard scatter
Directory-based	Lookup table: $\text{userId} \rightarrow \text{shardId}$	Maximum flexibility	Lookup table is a bottleneck
Geo-based	Users in US → Shard 1, EU → Shard 2	Low latency for geographic users	Imbalanced if one region grows faster

Sharding Challenges:

- **Cross-shard queries:** Joining data across shards requires application-level aggregation — extremely complex.
- **Rebalancing:** Adding a new shard requires migrating data from existing shards — requires careful planning (consistent hashing minimizes rebalancing).
- **Distributed transactions:** Cross-shard ACID transactions are near-impossible; must use eventual consistency or saga pattern.
- **Operational complexity:** Managing multiple DB instances, backups, failovers across shards.

When to Shard: Sharding is a **last resort**. Exhaust these first:

1. Query optimization (indexes, query rewriting).
2. Vertical scaling (bigger DB instance).
3. Read replicas (offload reads).
4. Caching (Redis cache for frequent reads).
5. Archiving (move old data to cold storage).

Real-Life Analogy

Read replicas are like a **library with one original catalog and several photocopied catalogs** distributed across the building. All new books are added to the original (primary). Patrons looking up existing books can use any photocopy (replica) — faster, no queue. But if you just added a book 2 minutes ago, the photocopy might not have it yet (replication lag).

Sharding is like **splitting the library into multiple buildings by topic**: Building A has Science & Math, Building B has History & Arts, Building C has Fiction. Each building is fully independent. Finding books within a topic is fast. But finding a book "about the history of mathematics" (cross-shard query) requires visiting both buildings and combining results manually.

Interview Context & "Why it Matters"

Database scaling is a centerpiece of system design interviews. Candidates who jump straight to sharding without discussing read replicas and caching show immaturity. The progression: optimize → cache → replicate → shard is the expected answer.

Production-Ready Code Snippet

```
java
```

```

import org.springframework.jdbc.datasource.lookup.AbstractRoutingDataSource;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.transaction.annotation.Transactional;
import javax.sql.DataSource;
import java.util.Map;
import java.util.concurrent.atomic.AtomicInteger;

// — Read/Write Routing DataSource —————

enum DataSourceType { PRIMARY, REPLICA }

class DataSourceContextHolder {
    private static final ThreadLocal<DataSourceType> context = new ThreadLocal<>();

    public static void setPrimary() { context.set(DataSourceType.PRIMARY); }
    public static void setReplica() { context.set(DataSourceType.REPLICA); }
    public static void clear() { context.remove(); }

    public static DataSourceType get() {
        return context.get() == null ? DataSourceType.PRIMARY : context.get();
    }
}

/**
 * Routes queries to primary (writes) or replica (reads) based on thread-local context
 * Activated by @Transactional(readOnly=true) – Spring sets a readOnly flag
 * which we intercept in lookupCurrentLookupKey.
 */
public class ReadWriteRoutingDataSource extends AbstractRoutingDataSource {

    // Round-robin across multiple replicas
    private final AtomicInteger replicaIndex = new AtomicInteger(0);
    private final int replicaCount;

    public ReadWriteRoutingDataSource(int replicaCount) {
        this.replicaCount = replicaCount;
    }

    @Override
    protected Object determineCurrentLookupKey() {
        DataSourceType type = DataSourceContextHolder.get();
        if (type == DataSourceType.REPLICA && replicaCount > 0) {

```

```

        // Round-robin across replicas
        int idx = replicaIndex.getAndIncrement() % replicaCount;
        return "replica-" + idx;
    }
    return "primary";
}
}

@Configuration
public class DataSourceConfig {

    @Bean
    public DataSource routingDataSource() {
        ReadWriteRoutingDataSource routing = new ReadWriteRoutingDataSource(2);

        // Configure primary
        com.zaxxer.hikari.HikariDataSource primary = buildDataSource(
            "jdbc:postgresql://primary-db:5432/appdb", "app_user", "secret"
        );

        // Configure replicas (using different hosts)
        com.zaxxer.hikari.HikariDataSource replica0 = buildDataSource(
            "jdbc:postgresql://replica0-db:5432/appdb", "app_readonly", "secret"
        );
        com.zaxxer.hikari.HikariDataSource replica1 = buildDataSource(
            "jdbc:postgresql://replica1-db:5432/appdb", "app_readonly", "secret"
        );

        routing.setTargetDataSources(Map.of(
            "primary", primary,
            "replica-0", replica0,
            "replica-1", replica1
        ));
        routing.setDefaultTargetDataSource(primary);
        return routing;
    }

    private com.zaxxer.hikari.HikariDataSource buildDataSource(String url, String user, String pass) {
        com.zaxxer.hikari.HikariConfig config = new com.zaxxer.hikari.HikariConfig();
        config.setJdbcUrl(url);
        config.setUsername(user);
        config.setPassword(pass);
        config.setMaximumPoolSize(20);
        config.setMinimumIdle(5);
    }
}

```



```
        config.setConnectionTimeout(3000); // 3 seconds
        config.setIdleTimeout(600_000);    // 10 minutes
        config.setMaxLifetime(1_800_000); // 30 minutes
        return new com.zaxxer.hikari.HikariDataSource(config);
    }
}
```

// — Service Layer: readOnly=true routes to replica —————

```
import org.springframework.stereotype.Service;

@Service
class ProductSearchService {

    private final ProductRepo productRepo;

    ProductSearchService(ProductRepo productRepo) {
        this.productRepo = productRepo;
    }

    // readOnly=true → Spring marks transaction as read-only
    // Our routing DataSource sends this to a read replica
    @Transactional(readOnly = true)
    public java.util.List<String> searchProducts(String keyword) {
        DataSourceContextHolder.setReplica();
        try {
            return productRepo.searchByKeyword(keyword);
        } finally {
            DataSourceContextHolder.clear();
        }
    }

    // Default (write transaction) → routed to primary
    @Transactional
    public void updateProductPrice(Long id, double price) {
        DataSourceContextHolder.setPrimary();
        try {
            productRepo.updatePrice(id, price);
        } finally {
            DataSourceContextHolder.clear();
        }
    }
}
```

```
interface ProductRepo {  
    java.util.List<String> searchByKeyword(String keyword);  
    void updatePrice(Long id, double price);  
}
```

Common Interview Question & Expert Answer

Q: "Design the database layer for a social media platform with 100M users. How would you approach scaling?"

A: I'd apply a layered approach:

Phase 1 — Vertical + Single Primary: Start with a well-tuned PostgreSQL on a large instance (e.g., `r6i.8xlarge`), 32 vCPUs, 256 GB RAM). Optimize queries, add indexes. Handles tens of millions of users.

Phase 2 — Read Replicas: Most social media reads (viewing feeds, profile pages, media) vastly outnumber writes. Add 3-5 read replicas. Route all SELECT queries (profile reads, feed reads, search) to replicas. Primary handles only writes (posts, follows, likes). This multiplies read capacity by 5x.

Phase 3 — Caching Layer: 80% of reads hit the same 20% of data (hot users, trending posts). Add Redis:

- **User profile cache:** `user:{id}` → 1 hour TTL.
- **Feed cache:** `feed:{userId}` → pre-computed fan-out on write.
- **Trending posts:** Sorted set by like count, refreshed every 60 seconds. This alone eliminates 70-80% of DB reads.

Phase 4 — Functional Sharding (Before Hash Sharding): Split by domain: User data in PostgreSQL, Post data in Cassandra (optimized for time-series append), Media metadata in DynamoDB, Social graph in Neo4j. Each service owns its data store — this is the microservices approach.

Phase 5 — Hash-Based User Sharding (At extreme scale): If the user table alone exceeds what one PostgreSQL instance can handle (~500M rows with large JSONB profiles), shard by `hash(userId) % N`. Consistent hashing minimizes data movement as shards are added.

Key Metrics to Decide:

- 10K writes/sec on primary → consider sharding writes.
- 1M rows in a single table → evaluate partitioning (PostgreSQL table partitioning by date or ID range).
- p99 query latency > 100ms → cache first, then replicate.

7.4 Caching Layers — Redis & Memcached

Clear Definition & Internal Mechanics

Caching stores the result of expensive computations (DB queries, API calls, computation) in a fast-access in-memory store. The cache sits between the application and the data source, serving cached results when available.

Redis is an in-memory **data structure store** supporting strings, hashes, lists, sets, sorted sets, bitmaps, streams, and HyperLogLog. It supports **persistence** (RDB snapshots, AOF log), **replication**, **Lua scripting**, **pub/sub**, and **transactions** (`(MULTI/EXEC)`).

Memcached is a simpler in-memory key-value store: strings only, no persistence, no replication, no complex data types. Extremely fast and memory-efficient. Multi-threaded (vs. Redis's event-loop single-thread model, though Redis 6.0 added I/O threading).

Feature	Redis	Memcached
Data types	Rich (list, set, sorted set, hash, stream)	String only
Persistence	✓ RDB + AOF	✗ Memory only
Replication	✓ Master-replica	✗ No
Clustering	✓ Redis Cluster	✓ via client
Pub/Sub	✓	✗
Transactions	✓ MULTI/EXEC	✗
Threading	Single-threaded event loop + I/O threads	Multi-threaded

Cache Strategies:

Strategy	Pattern	Use Case
Cache-Aside (Lazy)	Check cache → if miss, query DB → write to cache	General purpose, flexible
Write-Through	Write to cache AND DB synchronously	Strong consistency; slower writes
Write-Behind (Write-Back)	Write to cache → async write to DB	High write throughput; risk of data loss
Read-Through	Cache automatically loads from DB on miss	Simpler application code

Cache Eviction Policies (Redis):

- `allkeys-lru`: Evict least recently used keys from any key.
- `volatile-lru`: Evict LRU keys among those with a TTL set.
- `allkeys-lfu`: Evict least frequently used (Redis 4.0+).
- `noeviction`: Return error when memory full (useful for primary data stores).

Cache Stampede (Thundering Herd): When a popular cached key expires, hundreds of requests simultaneously miss the cache and all hit the DB to recompute the value. Prevention:

- Probabilistic early expiration (refresh before TTL expires).
- **Mutex lock**: First miss acquires a lock, computes, writes to cache. Other misses wait. (Redis `SETNX` / `SET NX EX`).
- **Redis Lua script** for atomic check-and-set.

Cache Penetration: Requests for non-existent keys bypass the cache (always miss) and hit the DB. Example: attacker queries random user IDs. Prevention: cache null results with a short TTL; use a **Bloom Filter** (probabilistic data structure) to quickly reject non-existent keys.

Cache Avalanche: Many cache keys expire simultaneously → DB overwhelmed. Prevention: randomize TTLs (`TTL = base ± random(0, 300 seconds)`).

Real-Life Analogy

A cache is a **desk drawer** next to your workstation. Your filing cabinet (database) has everything but is 30 feet away. You keep frequently used files in your desk drawer (cache). When you need a file, check the drawer first (cache hit → fast). If not there (cache miss), walk to the cabinet, get the file, and put a copy in the drawer for next time. The drawer has limited space, so you periodically clean out old files (eviction). The risk: your drawer copy might be slightly outdated if someone updated the filing cabinet while you weren't looking (cache staleness).

Interview Context & "Why it Matters"

Caching is one of the most impactful performance optimizations in distributed systems. The challenge isn't implementing it — it's managing **cache invalidation** (famously one of the two hard problems in computer science). Interviewers probe: when to use TTL vs. explicit invalidation, how to handle stampede, and the consistency tradeoffs.

Production-Ready Code Snippet

```
java
```

```

import org.springframework.cache.annotation.*;
import org.springframework.cache.CacheManager;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.cache.RedisCacheConfiguration;
import org.springframework.data.redis.cache.RedisCacheManager;
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.serializer.*;
import org.springframework.stereotype.Service;
import java.time.Duration;
import java.util.Map;
import java.util.Optional;

// — Redis Cache Configuration —————

@Configuration
@EnableCaching
public class CacheConfig {

    @Bean
    public CacheManager cacheManager(RedisConnectionFactory connectionFactory) {

        // Default: 10-minute TTL, JSON serialization (not Java serialization – port
        RedisCacheConfiguration defaultConfig = RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(10))
            .serializeValuesWith(RedisSerializationContext.SerializationPair
                .fromSerializer(new GenericJackson2JsonRedisSerializer()))
            .disableCachingNullValues();

        // Per-cache TTL overrides
        Map<String, RedisCacheConfiguration> cacheConfigs = Map.of(
            "products",          defaultConfig.entryTtl(Duration.ofHours(1)),
            "user-profiles",     defaultConfig.entryTtl(Duration.ofMinutes(30)),
            "search-results",    defaultConfig.entryTtl(Duration.ofMinutes(5)),
            "rate-limits",       defaultConfig.entryTtl(Duration.ofSeconds(60))
        );

        return RedisCacheManager.builder(connectionFactory)
            .cacheDefaults(defaultConfig)
            .withInitialCacheConfigurations(cacheConfigs)
            .enableStatistics() // exposes cache hit/miss metrics
            .build();
    }
}

```

```

}

// — Service with Caching Annotations —————

@Service
public class ProductCacheService {

    private final ProductRepository productRepository;

    public ProductCacheService(ProductRepository productRepository) {
        this.productRepository = productRepository;
    }

    // Cache-Aside: Spring checks Redis before calling this method.
    // Cache key: "products::123" (cacheName::key)
    @Cacheable(value = "products", key = "#id",
        condition = "#id > 0", // only cache valid IDs
        unless = "#result == null" // don't cache null (penetration p
    public ProductDto getProduct(Long id) {
        if (id == null || id <= 0) throw new IllegalArgumentException("ID must be po
        System.out.println("[DB] Loading product " + id + " from database.");
        return productRepository.findById(id)
            .map(p -> new ProductDto(p.getId(), p.getName(), p.getPrice(), p.getCate
            .orElse(null);
    }

    // Cache eviction: when product is updated, remove stale cache entry
    @CacheEvict(value = "products", key = "#id")
    @Transactional
    public void updateProduct(Long id, double newPrice) {
        if (id == null || id <= 0) throw new IllegalArgumentException("ID must be po
        if (newPrice < 0) throw new IllegalArgumentException("Price cannot be negati
        productRepository.updatePrice(id, newPrice);
        // Cache entry is evicted AFTER the DB update succeeds
    }

    // Evict entire cache (e.g., after a bulk update)
    @CacheEvict(value = "products", allEntries = true)
    public void invalidateAllProducts() {
        System.out.println("[Cache] Evicting all product cache entries.");
    }

    // Cache update: update cache entry AND DB atomically
    @CachePut(value = "products", key = "#result.id()")

```

```

@Transactional
public ProductDto createProduct(String name, double price, String category) {
    // Saves to DB and writes result to cache – cache is always warm after creat
    ProductDto created = new ProductDto(100L, name, price, category);
    System.out.println("[Cache] Warming cache for new product: " + created.id())
    return created;
}
}

record ProductDto(Long id, String name, double price, String category) {}
interface ProductRepository {
    Optional<Product> findById(Long id);
    void updatePrice(Long id, double price);
}
class Product {
    Long id; String name; double price; String category;
    Long getId() { return id; } String getName() { return name; }
    double getPrice() { return price; } String getCategory() { return category; }
}

```

Common Interview Question & Expert Answer

Q: "What is cache invalidation and why is it the hard problem? How do you handle it in practice?"

A: Cache invalidation is the problem of ensuring the cache stays consistent with the source of truth (database) when the underlying data changes. It's "hard" because:

1. **Distributed state:** In a horizontally scaled system, the cache is a separate distributed system from the DB. Updates to the DB don't automatically propagate to the cache.
2. **Atomicity:** Updating the DB and invalidating the cache in one atomic operation is impossible across two separate systems. Failures between the DB write and cache eviction leave stale data indefinitely.
3. **Concurrency:** Race condition: Thread A reads from DB (miss), Thread B updates DB + evicts cache, Thread A writes old value to cache → stale entry appears right after invalidation.

In practice, I use these strategies:

1. **TTL-based expiry** (simplest): Every cache entry expires after N seconds. Stale data exists for at most N seconds. Acceptable for non-critical data (product descriptions, config).
2. **Write-through + eviction on update:** After DB write succeeds, immediately call `cache.evict(key)`. Risk: if eviction fails, stale data lingers until TTL. Mitigation: short TTL

as a safety net.

3. **Event-driven invalidation:** DB writes publish events (Kafka, Redis Pub/Sub, Debezium CDC). A cache invalidation service consumes events and evicts cache entries. Decoupled, eventually consistent.
 4. **Versioned cache keys:** Include a version number in the cache key (`product:123:v42`). On update, increment the version in DB. Next cache lookup uses `product:123:v43` — always a miss, so fresh data is fetched. Old keys expire by TTL. No explicit eviction needed, but cache memory grows until TTL cleanup.
 5. **Read-your-writes guarantee:** After a user writes data, route their subsequent reads to the primary DB (bypassing cache) for 30 seconds. This prevents the user from seeing stale data after their own update.
-

7.5 Asynchronous Messaging Queues — Kafka & RabbitMQ

Clear Definition & Internal Mechanics

Message queues decouple producers (services that generate work) from consumers (services that process work). Instead of synchronous HTTP calls (consumer must be available immediately), the producer writes a message to the queue and continues. The consumer processes it at its own pace.

Apache Kafka — Distributed Event Streaming Platform

Architecture:

- **Topics:** Named append-only logs. Messages are written to the end of the log.
- **Partitions:** Each topic is split into N partitions. Each partition is an independent ordered log. Parallelism = number of partitions.
- **Brokers:** Kafka cluster nodes. Each broker stores some partitions (replicated across brokers for fault tolerance).
- **Producers:** Write records to a topic (optionally specifying a partition key for ordering guarantees).
- **Consumers:** Read records from partitions. Each consumer group receives each message exactly once. Multiple consumers in a group parallelize processing.
- **Offsets:** Each message in a partition has a unique sequential offset. Consumers commit offsets to track their position. On restart, processing resumes from the last committed offset.
- **Retention:** Messages are retained for a configurable period (e.g., 7 days) regardless of consumption — unlike queues where messages are deleted after acknowledgment. This enables **replay**, **audit logs**, and multiple consumer groups reading the same topic independently.

Ordering Guarantee: Within a partition, messages are strictly ordered. Across partitions, there is no ordering guarantee. To maintain ordering for related messages (e.g., all events for one user), use the user ID as the partition key — all messages for that user land in the same partition.

Throughput: Kafka achieves millions of messages/second via batching, sequential disk I/O (OS page cache), zero-copy (`sendfile()`), and compression.

RabbitMQ — Traditional Message Broker (AMQP)

Architecture:

- **Exchanges:** Receive messages from producers and route to queues based on routing rules.
 - `direct`: Route by exact routing key.
 - `topic`: Route by wildcard pattern (`order.*`, `*.failed`).
 - `fanout`: Broadcast to all bound queues.
 - `headers`: Route by message headers.
- **Queues:** Stores messages. Consumed FIFO. Message is deleted after acknowledgment.
- **Bindings:** Rules connecting exchanges to queues.
- **Acknowledgments:** Consumer sends `ack` after successful processing. Broker deletes the message. If consumer dies without acking, message is re-queued.

Comparison:

Dimension	Kafka	RabbitMQ
Model	Distributed log (pull)	Message broker (push)
Message retention	Days/weeks (configurable)	Until consumed + acked
Ordering	Per-partition	Per-queue (FIFO)
Throughput	Millions/sec	~50K/sec per queue
Replay	✅ Yes (re-read old offsets)	❌ No
Use case	Event streaming, audit, CDC	Task queues, RPC, routing
Consumers	Pull-based	Push-based

Use Kafka when: Event sourcing, audit trails, CDC (Change Data Capture), high-throughput event streams, multiple consumers needing the same events (analytics + notifications + audit simultaneously).

Use RabbitMQ when: Task queues (image processing, email sending), complex routing (topic exchanges), RPC patterns, work distribution with priority queues, low-message-volume inter-service communication.

Real-Life Analogy

Kafka is like a **newspaper distribution system**: the printing press (producer) prints thousands of papers (messages) onto an endless conveyor belt (topic/partition). Subscribers (consumers) can read any paper from any point in time — today's, yesterday's, or last week's (replay). Multiple subscriber groups can read independently.

RabbitMQ is like a **post office sorting room**: letters arrive (messages), the sorter (exchange) reads the address (routing key) and drops each letter into the right mailbox (queue). The recipient collects and signs for the letter (ack). Once signed for, the post office discards its copy.

Interview Context & "Why it Matters"

Async messaging is a core pattern for resilient, scalable microservices. It enables: decoupling (producer doesn't need consumer to be available), load leveling (absorbs traffic spikes), retry on failure (re-queue nacked messages), and fan-out (one event → many consumers). Interviewers probe: at-least-once vs. exactly-once semantics, offset management, consumer group scaling, and dead-letter queues.

Production-Ready Code Snippet

```
java
```

```
import org.springframework.kafka.annotation.KafkaListener;
import org.springframework.kafka.core.KafkaTemplate;
import org.springframework.kafka.support.SendResult;
import org.springframework.stereotype.Service;
import org.springframework.stereotype.Component;
import java.util.concurrent.CompletableFuture;

// — Kafka Producer —————

@Service
public class OrderEventProducer {

    private static final String TOPIC = "order-events";
    private final KafkaTemplate<String, OrderEvent> kafkaTemplate;

    public OrderEventProducer(KafkaTemplate<String, OrderEvent> kafkaTemplate) {
        this.kafkaTemplate = kafkaTemplate;
    }

    /**
     * Sends order event to Kafka.
     * Key = orderId → ensures all events for one order go to the same partition (or
     * Returns a CompletableFuture for async confirmation.
     */
    public CompletableFuture<SendResult<String, OrderEvent>> publishOrderEvent(OrderEvent event) {
        if (event == null || event.orderId() == null) {
            throw new IllegalArgumentException("OrderEvent and orderId must not be null");
        }

        CompletableFuture<SendResult<String, OrderEvent>> future =
            kafkaTemplate.send(TOPIC, event.orderId(), event).toCompletableFuture();

        future.whenComplete((result, ex) -> {
            if (ex != null) {
                System.err.printf("[Kafka] Failed to send event for order %s: %s\n",
                    event.orderId(), ex.getMessage());
                // In production: publish to a dead-letter topic or alert
            } else {
                System.out.printf("[Kafka] Sent order %s → partition %d, offset %d\n",
                    event.orderId(),
                    result.getRecordMetadata().partition(),
                    result.getRecordMetadata().offset());
            }
        });
    }
}
```

```

    });

    return future;
}
}

record OrderEvent(String orderId, String status, double totalAmount, java.time.Insta

// — Kafka Consumer —————

@Component
public class OrderEventConsumer {

    // groupId: all instances in the same group share partitions – each message prod
    // Each instance in the group gets assigned 1+ partitions for parallelism
    @KafkaListener(
        topics = "order-events",
        groupId = "notification-service",
        containerFactory = "kafkaListenerContainerFactory"
    )
    public void consumeOrderEvent(
        org.apache.kafka.clients.consumer.ConsumerRecord<String, OrderEvent> rec

        OrderEvent event = record.value();
        if (event == null) {
            System.err.println("[Kafka] Received null event, skipping.");
            return;
        }

        System.out.printf("[Consumer] Partition=%d Offset=%d Order=%s Status=%s%n",
            record.partition(), record.offset(),
            event.orderId(), event.status());

        try {
            processEvent(event);
            // With enable.auto.commit=false (recommended), Spring Kafka commits off
            // after this method returns successfully.
            // If an exception is thrown, offset is NOT committed → message is repro
        } catch (Exception e) {
            System.err.printf("[Consumer] Failed to process order %s: %s%n",
                event.orderId(), e.getMessage());
            // Options: throw to trigger retry (with retry listener), or send to DLT
            throw new org.springframework.kafka.listener.ListenerExecutionFailedExce
                "Processing failed for order: " + event.orderId(), e);

```

```

    }
}

private void processEvent(OrderEvent event) {
    // Send email, update analytics, trigger fulfillment, etc.
    System.out.println("[Notification] Processing status change: " + event.status);
}
}

// — Kafka Configuration —————

// application.yaml equivalent:
// spring:
//   kafka:
//     bootstrap-servers: kafka-broker:9092
//     producer:
//       key-serializer: org.apache.kafka.common.serialization.StringSerializer
//       value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
//       acks: all # Wait for all replicas to ack (durability)
//       retries: 3
//       enable.idempotence: true # Exactly-once delivery semantics
//     consumer:
//       group-id: notification-service
//       auto-offset-reset: earliest # Start from beginning on first run
//       enable-auto-commit: false # Manual offset commit (at-least-once)
//       key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
//       value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
//     listener:
//       ack-mode: record # Commit after each record

```

Common Interview Question & Expert Answer

Q: "What is the difference between at-most-once, at-least-once, and exactly-once delivery semantics in Kafka?"

A:

At-Most-Once: Messages may be lost, never duplicated. Offset is committed *before* processing. If processing fails after commit, the message is gone.

```

Receive → Commit Offset → Process
                        ↑ crash here → message lost forever

```

Use case: Metrics/telemetry where some loss is acceptable.

At-Least-Once (most common): Messages are never lost, but may be duplicated. Offset committed *after* successful processing. If the consumer crashes after processing but before committing, the message is reprocessed.

Receive → Process → Commit Offset
↑ commit failure → reprocessed → possible duplicate

Use case: Email notifications, fulfillment — handle duplicates by making consumers **idempotent** (re-processing the same message produces the same result).

Exactly-Once: Every message processed exactly once. Requires:

1. Producer idempotence (`enable.idempotence=true`): Kafka deduplicates producer retries using a sequence number per producer-partition pair.
2. Kafka Transactions: Producer wraps produces + consumer offset commits in an atomic transaction. If the transaction aborts, both the message write and offset commit are rolled back.
3. Consumer reads with (`isolation.level=read_committed`): Ensures consumers only see messages from committed transactions.

Trade-off: Exactly-once has 10–20% overhead compared to at-least-once. For most business systems, **at-least-once + idempotent consumers** is the right choice — simpler, lower overhead, and practically indistinguishable from exactly-once when consumers handle duplicates via idempotency keys.

How to make a consumer idempotent: Store a unique event ID (`orderId` + `eventType` + `eventTimestamp`) in a `processed_events` DB table with a unique constraint. Before processing: check if ID exists → skip. After processing: insert ID. This is the **inbox pattern**.

7.6 The CAP Theorem

Clear Definition & Internal Mechanics

The **CAP Theorem** (Brewer's Theorem, 2000) states that a distributed data store can provide **at most two** of the following three guarantees simultaneously:

- **C — Consistency:** Every read receives the most recent write or an error. All nodes see the same data at the same time.
- **A — Availability:** Every request receives a non-error response (though the response may not contain the most recent data). The system is always operational.

- **P — Partition Tolerance:** The system continues operating even when the network partition (message loss or delay between nodes) occurs.

The Critical Insight — Partition Tolerance is Non-Optional: In any real distributed system, network partitions **will** occur (hardware failure, network congestion, datacenter outages). Therefore, **P is not truly optional** — you must tolerate partitions. The real choice is between **C and A** during a partition:

- **CP (Consistency + Partition Tolerance):** When a partition occurs, the system refuses to serve requests (returns errors or waits) rather than risk returning stale data. It sacrifices Availability.
 - Examples: ZooKeeper (distributed coordination), etcd (Kubernetes config store), HBase, MongoDB (default with `majority` read concern).
- **AP (Availability + Partition Tolerance):** When a partition occurs, the system continues serving requests with potentially stale data. It sacrifices Consistency (offers **eventual consistency** — nodes converge when the partition heals).
 - Examples: Cassandra, DynamoDB (default), CouchDB, DNS.

PACELC Extension (More Nuanced): CAP only addresses behavior during partitions. Bailis et al.'s PACELC theorem adds the normal-operation trade-off:

- During a **Partition** (P): choose between **Availability** and **Consistency**.
- Else (no partition): choose between **Latency** and **Consistency**.

A system like Cassandra is PA/EL: Availability during partitions, Low latency in normal operation (with eventual consistency). Spanner is PC/EC: Consistent during partitions, Consistent in normal operation (with higher latency due to TrueTime synchronization).

Consistency Models (Spectrum):

Model	Guarantee	Example
Strict Linearizability	All ops appear instantaneous, globally ordered	Single-node RDBMS
Sequential Consistency	All nodes see ops in same order, not necessarily real-time	Zookeeper
Eventual Consistency	All replicas converge given no new writes	Cassandra, DynamoDB
Read Your Writes	After write, you always see your own write	DynamoDB sessions
Monotonic Reads	You never see older data after seeing newer data	Session-pinned replicas
Causal Consistency	Causally related ops appear in order	MongoDB causal sessions

Real-Life Analogy

Imagine a **multi-branch bank** during a phone network outage (partition).

CP approach (banks can't talk): The teller freezes all transactions and says "Sorry, our system is down. Please come back later." No mistakes, but service is unavailable.

AP approach: The teller continues serving customers using the last known account balance. After the network recovers, branches reconcile discrepancies. You might have overdrafted at branch B while your deposit at branch A was still in transit (stale reads) — but the bank never stopped operating.

Interview Context & "Why it Matters"

CAP Theorem is a foundational system design concept. Every data storage choice in a distributed system is an implicit CAP trade-off. Interviewers test whether candidates:

- Know which databases are CP vs AP.
- Can articulate why "choosing CA" is impossible in a distributed system.
- Can apply the theorem to real scenarios (banking = CP, shopping cart = AP).
- Know the PACELC extension for more nuanced analysis.

Production-Ready Code Snippet

```
java
```

```

/**
 * Demonstrates CAP trade-off at the application level using Spring Data.
 * Shows how to configure Cassandra (AP) vs PostgreSQL (CP) read consistency.
 */

// — AP System: Cassandra with tunable consistency —————

import com.datastax.oss.driver.api.core.ConsistencyLevel;
import org.springframework.data.cassandra.core.CassandraTemplate;
import org.springframework.data.cassandra.core.query.Query;
import org.springframework.stereotype.Repository;

// In Cassandra, you tune consistency per query:
// ConsistencyLevel.ONE → Read from 1 replica (low latency, may be stale = AP)
// ConsistencyLevel.QUORUM → Read from majority (higher latency, consistent = CP for
// ConsistencyLevel.ALL → Read from all replicas (highest consistency, lowest avail

@Repository
class ShoppingCartRepository {

    private final CassandraTemplate cassandraTemplate;

    ShoppingCartRepository(CassandraTemplate cassandraTemplate) {
        this.cassandraTemplate = cassandraTemplate;
    }

    // AP: Accept stale cart data for better availability and lower latency
    // A user sees their cart from 100ms ago – acceptable UX trade-off
    public Object findCartByUserId(String userId) {
        if (userId == null || userId.isBlank()) throw new IllegalArgumentException("
        // In real Cassandra query:
        // Statement.builder("SELECT * FROM shopping_carts WHERE user_id = ?")
        //     .setConsistencyLevel(ConsistencyLevel.ONE)
        //     .build()
        System.out.printf("[Cassandra/AP] Reading cart for user %s with ONE consisten
        return null; // simplified
    }

    // CP: Require quorum read for checkout – must see latest inventory/price
    public Object checkoutCart(String userId) {
        if (userId == null || userId.isBlank()) throw new IllegalArgumentException("
        // Statement with ConsistencyLevel.QUORUM – fails if < majority of replicas
        System.out.printf("[Cassandra/CP] Checkout for user %s with QUORUM consisten

```

```

        return null; // simplified
    }
}

// — CP System: PostgreSQL with serializable isolation —————

import org.springframework.transaction.annotation.Isolation;
import org.springframework.transaction.annotation.Transactional;
import org.springframework.stereotype.Service;

@Service
class BankTransferService {

    /**
     * Financial transfers: must be CP.
     * If the DB is partitioned from a replica, block rather than serve stale balance.
     * Serializable isolation prevents phantom reads – the strongest guarantee in RDMA.
     */
    @Transactional(isolation = Isolation.SERIALIZABLE)
    public void transfer(Long fromId, Long toId, double amount) {
        // In a PostgreSQL cluster: all reads/writes go to primary (CP behavior)
        // If primary is unreachable: SQLException → caller receives 503 (availability)
        System.out.printf("[PostgreSQL/CP] Transferring %.2f from %d to %d\n", amount, fromId, toId);
        // accountRepository.debit(fromId, amount);
        // accountRepository.credit(toId, amount);
    }
}

// — Application-level eventual consistency (AP pattern) —————

import org.springframework.scheduling.annotation.Async;

@Service
class UserProfileService {

    /**
     * User profile update: AP pattern.
     * Write to primary DB synchronously.
     * Cache invalidation + search index update asynchronously.
     * For a brief window, search index may show old data (eventual consistency).
     * Acceptable: users tolerate their profile bio update taking 5 seconds to appear.
     */
    @Transactional
    public void updateProfile(Long userId, String bio) {

```

```

        if (userId == null || bio == null) throw new IllegalArgumentException("Invalid user bio");
        // profileRepository.update(userId, bio); // synchronous, CP
        System.out.println("[Primary] Profile updated in DB.");

        // Async eventual consistency: non-critical secondary stores
        triggerSearchIndexUpdate(userId);
        invalidateProfileCache(userId);
    }

    @Async
    public void triggerSearchIndexUpdate(Long userId) {
        System.out.println("[Async/AP] Updating search index for user " + userId);
        // searchIndexClient.index(userId); // may lag by seconds
    }

    @Async
    public void invalidateProfileCache(Long userId) {
        System.out.println("[Async/AP] Invalidating cache for user " + userId);
        // cacheManager.getCache("user-profiles").evict(userId);
    }
}

```

Common Interview Question & Expert Answer

Q: "Can you choose CA (Consistency + Availability, without Partition Tolerance) in a distributed system?"

A: In theory, CA exists only for **single-node systems** (a standalone PostgreSQL on one server has no network partition possible — it's just you and the DB). The moment you add a second node, network partitions *will* happen, and you must handle them — making P non-optional in any real distributed system.

The claim "our system is CA" typically means one of two things:

1. The system is not truly distributed (single node).
2. The operators have chosen to stop the system entirely during partitions (which means they sacrificed A, making it CP in practice).

Real-world application:

- **RDBMS (PostgreSQL, MySQL) in a cluster:** CA in theory, but in practice configured as CP — replication lag + automatic failover means you sacrifice availability (election takes 10–30 seconds) to guarantee consistency.

- **DynamoDB:** AP by default (eventually consistent reads). Can opt into CP with `ConsistentRead=true` (higher cost + higher latency).
- **ZooKeeper:** CP — sacrifices availability during partition. A ZooKeeper quorum that loses majority ($N/2+1$ nodes) stops accepting writes until quorum is restored.

The practical takeaway for system design interviews: **Default to AP** for user-facing read-heavy services (shopping carts, social feeds, user profiles — where stale data for seconds is acceptable). **Choose CP** for anything involving money, inventory counts, or coordination (distributed locks, counters, financial ledgers).

Final Study Summary Table

Pillar	Sub-Topic	Key Pattern / Algorithm	Interview Priority
JVM	Stack Memory	Frame push/pop, local var isolation	★★★
JVM	Heap Memory	Generational GC, Eden→Survivor→Tenured	★★★★★
JVM	Metaspace	ClassLoader leak, native memory	★★★
JVM	Minor vs Full GC	STW, copy algorithm, mark-sweep-compact	★★★★★
JVM	G1GC vs ZGC	Region-based vs colored pointers	★★★★★
JVM	Reference Strengths	Soft/Weak/Phantom + ReferenceQueue	★★★
JVM	Memory Leaks	Static refs, ThreadLocal, inner class	★★★★★★
JVM	Tooling	jcmd, jstack, VisualVM, heap dumps	★★★★★
Spring	IoC / DI	Constructor injection, proxy mechanism	★★★★★★
Spring	ApplicationContext	Bean lifecycle, BeanPostProcessor	★★★★★
Spring	Bean Scopes	Singleton trap, ObjectProvider, @Lookup	★★★★★
Spring	Stereotypes	@Configuration proxy, classpath scan	★★★
Spring	MVC Workflow	DispatcherServlet pipeline, interceptors	★★★★★
Spring	REST APIs	HTTP semantics, status codes, idempotency	★★★★★★
Spring	ControllerAdvice	Exception handling, RFC 7807	★★★★★
Spring	AOP	Proxy, pointcut, around advice	★★★★★

Pillar	Sub-Topic	Key Pattern / Algorithm	Interview Priority
Spring	JPA / Hibernate	Entity states, dirty checking, fetch types	★★★★★★
Spring	Transactions	Propagation, isolation, rollback	★★★★★★
Spring	N+1 Problem	JOIN FETCH, @EntityGraph, @BatchSize	★★★★★★
Spring	Auto-Configuration	@Conditional, @EnableAutoConfiguration	★★★★
SysDesign	H vs V Scaling	Statelessness, distributed sessions	★★★★★★
SysDesign	Load Balancers	L4/L7, health checks, Nginx config	★★★★★
SysDesign	Read Replicas	Replication lag, routing DataSource	★★★★★★
SysDesign	Sharding	Shard key, consistent hashing	★★★★★
SysDesign	Caching	Cache-aside, eviction, stampede, avalanche	★★★★★★
SysDesign	Kafka / RabbitMQ	Partitions, consumer groups, semantics	★★★★★★
SysDesign	CAP Theorem	CP vs AP, PACELC, consistency models	★★★★★★

This document represents an exhaustive technical reference for Java SDE interview preparation. Each section is designed to be studied in depth and revisited before live interview rounds. Good luck — you've got this!